AHD-Jailbreak: Adaptive Hypergraph Defense for Detecting Evolving Jailbreak Attacks in Large Language Models

Complete Research Dossier, Mathematical Architecture, Verified Source Code & Dataset Directory, and Phased Implementation Plan — Department of Computer Science, Godavari Global University

Project Metadata Field	Official Review 0 Registration Details	System Scope & Target Deliverables
Student Researchers	1. Sesetti Pavan Kumar Swamy (PIN: 24555A0519, CSE – C) 2. Rajana Rajeswari (PIN: 23551A05C9, CSE – B)	Adaptive Hypergraph Construction, Spectral Message Passing, Online Weight/Edge Adaptation & Benchmark Evaluation
Faculty Guide	Mrs. S. Suguna Sri (Department of Computer Science, Godavari Global University)	Review 0 Concept Presentation, Working Prototype Demonstration & Final Thesis Supervision
Architecture Paradigm	Lightweight Model-Agnostic Pre-Inference Wrapper Layer	Intercepts unsafe prompts locally before routing to OpenAI, Anthropic, Gemini, or Llama endpoints

1. Executive Research Synthesis: Why Adaptive Hypergraphs Outperform Static & Pairwise Defenses

Large Language Models (LLMs) deployed in mission-critical applications remain highly vulnerable to adversarial jailbreak strategies that bypass safety alignment through roleplay persona adoption, payload encoding, hypothetical framing, and multi-turn conversational escalation. While first-generation guardrails such as Meta Llama Guard 3, Llama Prompt Guard 2, AllenAI WildGuard, and NVIDIA NeMo Guardrails inspect prompts as flat token sequences or fixed dialogue flows, recent empirical studies prove that static classifiers degrade sharply against evolving, multi-turn attack trajectories. In LLM Defenses Are Not Robust to Multi-Turn Human Jailbreaks Yet (Li et al., 2024), multi-turn human red-teaming achieved over 70% Attack Success Rate (ASR) on HarmBench against defenses that reported single-digit ASRs on automated single-turn attacks.

Recent graph-based defenses such as GuardNet: Graph-Attention Filtering for Jailbreak Defense (Forough et al., 2025) and G-Guard: Attention-Aware GNN-based Input Defense against Multi-Turn LLM Jailbreak (Huang et al., 2025) model syntactic token dependencies or pairwise query-keyword relationships using standard Graph Neural Networks (GNNs). However, standard graphs restrict edges to pairwise interactions ($e = (u, v)$). In real-world jailbreaks, an individual prompt turn or keyword is frequently benign in isolation (for example, asking about network socket scripts for a university lab vs. combining a fictional thriller persona, a multi-turn escalation cue, an encoding instruction, and a

restricted cybersecurity policy target). Only the **joint higher-order co-occurrence** of three or more nodes across prompt, conversational context, semantic framing, and safety policy dimensions reveals the malicious intent.

AHD-Jailbreak addresses both limitations simultaneously. First, it formulates the safety representation as a **heterogeneous hypergraph** where each hyperedge connects an arbitrary subset of nodes across five distinct modalities. Second, unlike static hypergraphs whose incidence structure is frozen after offline training, AHD-Jailbreak incorporates an **online structural anomaly detector and adaptive updater** that adjusts hyperedge risk weights and dynamically spawns new hyperedges when unseen attack compositions are encountered. Placed as a lightweight pre-inference wrapper in front of any LLM API, AHD-Jailbreak blocks policy-violating requests in under 2 milliseconds locally—eliminating unnecessary downstream LLM inference costs.

---

2. Literature Survey & Comparative Baseline Positioning

To establish a rigorous academic foundation for Review 0 and your final thesis document, the table below synthesizes the foundational hypergraph learning papers, state-of-the-art GNN jailbreak filters, and industrial guardrail baselines against which AHD-Jailbreak is positioned.

Paper / System	Core Mechanism & Representation	Key Limitation Addressed by AHD-Jailbreak	Primary Reference Link
HGNN (Feng et al., AAAI 2019)	Spectral hypergraph convolution using incidence matrix H and normalized Laplacian operator.	Assumes a fixed, static hypergraph topology constructed offline prior to inference.	arXiv:1809.09401 | GitHub Code
DHGNN (Jiang et al., IJCAI 2019)	Alternates dynamic hypergraph construction (k-NN / k-Means clustering) with hypergraph convolution.	Designed for general vision/citation classification; lacks safety policy anchors and online streaming threat updates.	IJCAI 2019 Paper | GitHub Code
HCHA (Bai et al., Pattern Recognition 2021)	Introduces hypergraph attention where incidence weights between nodes and hyperedges are learned via attention.	Does not spawn new hyperedges online when zero-day adversarial patterns appear in streaming prompts.	arXiv:1901.08150
GuardNet (Forough et al., 2025)	Hierarchical pairwise GNN combining sequential links, syntactic dependencies, and token attention.	Pairwise edges cannot natively capture 4-way or 5-way multi-turn context-policy-framing interactions; static weights.	arXiv:2509.23037

Paper / System	Core Mechanism & Representation	Key Limitation Addressed by AHD-Jailbreak	Primary Reference Link
G-Guard (Huang et al., 2025)	Pairwise entity graph over multi-turn queries and harmful keywords with single-turn query retrieval augmentation.	Depends on keyword-entity pairwise matching and static GNN parameters; vulnerable to unseen obfuscated framing.	arXiv:2507.07146
Llama Guard 3 / Prompt Guard 2 (Meta, 2024–2025)	Fine-tuned Transformer safety classifiers (22M to 8B parameters) scoring flat prompt/response text.	High GPU inference cost for 8B models; static weights suffer recall drops on multi-turn crescendo jailbreaks.	Llama Guard 3 | Prompt Guard 2
WildGuard (AllenAI, 2024)	7B open moderation model trained on WildGuardMix for prompt harmfulness, response harmfulness, and refusal detection.	Requires full 7B LLM forward pass per turn and cannot adapt online to new attack templates without retraining.	arXiv:2406.18495 | GitHub Code

3. Mathematical Formulation & System Architecture of AHD-Jailbreak

The proposed AHD-Jailbreak framework operates as a modular four-stage pipeline: (1) Multi-Modal Feature & Node Extraction, (2) Spectral Hypergraph Message Passing, (3) Pre-Inference Risk & Structural Anomaly Scoring, and (4) Online Adaptive Hypergraph Evolution.

3.1 Heterogeneous Hypergraph Definition

At conversational turn t, AHD-Jailbreak maintains an adaptive weighted hypergraph $G_t = (V_t, E_t, W_t)$. Unlike a conventional graph, the vertex set V_t is partitioned into five functional node families:

$$V_t = V_{prompt} \cup V_{context} \cup V_{semantic} \cup V_{policy} \cup V_{pattern}$$

- **Prompt Nodes (V_{prompt})**: Dense semantic representations of the active user utterance p_t extracted via Sentence-Transformers (`all-MiniLM-L6-v2`, 384-D projected to $d = 64$).
- **Conversational Context Nodes ($V_{context}$)**: State nodes capturing dialogue trajectory across prior turns $C_{t-1} = \{p_1, \ldots, p_{t-1}\}$, including `ctx:single_turn`, `ctx:benign_dialogue`, and `ctx:multi_turn_escalating` (measured by cosine semantic drift toward restricted policy centroids across consecutive turns).
- **Semantic Feature Nodes ($V_{semantic}$)**: Pragmatic and structural intent indicators including persona adoption (`sem:roleplay_persona`), instruction override (`sem:rule_override`), encoding/cipher obfuscation (`sem:obfuscation_encoding`), hypothetical/fictional distancing (`sem:hypothetical_framing`), stepwise escalation (`sem:stepwise_escalation`), and educational/defensive inquiry (`sem:benign_technical`).
- **Safety Policy Constraint Nodes (V_{policy})**: Canonical safety taxonomy anchors aligned with JailbreakBench and Llama Guard categories (cyber exploitation, chemical/biological weapons,

financial fraud/phishing, physical harm/sabotage, privacy violation).

- **Emerging Attack Pattern Nodes ($V_{pattern}$)**: Prototype centroids representing known jailbreak families (e.g., DAN overrides, Base64 smuggling, multi-turn crescendo) and newly discovered cluster centroids.

3.2 Incidence Matrix & Spectral Hypergraph Convolution

Each hyperedge $e \in E_t$ connects an arbitrary subset of vertices $e \subseteq V_t$ with cardinality $|e| \geq 2$. The structure is encoded in the binary (or attention-weighted) incidence matrix $H_t \in [0, 1]^{|V_t| \times |E_t|}$, where $H_t(v, e) = 1$ if vertex v participates in hyperedge e. Let $W_t = diag(w(e_1), \ldots, w(e_{|E_t|}))$ denote the diagonal matrix of adaptive hyperedge risk weights. The vertex degree of $v \in V_t$ and hyperedge degree of $e \in E_t$ are defined as:

$$d(v) = \Sigma_{e \in E_t} w(e)\, H_t(v, e), \qquad \delta(e) = \Sigma_{v \in V_t} H_t(v, e)$$

Let $D_v = diag(d(v_1), \ldots, d(v_{|V_t|}))$ and $D_e = diag(\delta(e_1), \ldots, \delta(e_{|E_t|}))$. Following Feng et al. (AAAI 2019), node embeddings $X^{(l)} \in R^{|V_t| \times d_l}$ propagate across high-order hyperedges via the normalized spectral hypergraph convolution layer:

$$X^{(l+1)} = \sigma(\, D_v^{-1/2}\, H_t\, W_t\, D_e^{-1}\, H_t^{\top}\, D_v^{-1/2}\, X^{(l)}\, \Theta^{(l)}\,)$$

This two-phase node-to-hyperedge-to-node aggregation ($H_t^{\top}$ aggregates node features into hyperedge representations, and H_t redistributes weighted hyperedge context back to constituent nodes) ensures that any policy constraint node v_{pol} co-occurring with subtle multi-turn escalation and framing cues receives amplified risk signals.

3.3 Pre-Inference Risk & Unseen Structural Anomaly Scoring

For an incoming prompt p_t activating vertex subset $A_t \subseteq V_t$, AHD-Jailbreak computes a composite risk score $R(p_t) \in [0, 1]$ prior to LLM routing:

$$R(p_t) = \lambda_1\, S_{edge}(A_t) + \lambda_2\, S_{spec}(A_t) + \lambda_3\, S_{anom}(A_t)$$

- **Hyperedge Alignment Score (S_{edge})**: Sum of hyperedge weights $w(e)$ weighted by the overlap ratio $|A_t \cap e| / |A_t|$ across matched unsafe hyperedges.
- **Spectral Subspace Risk (S_{spec})**: Cosine/linear risk projection of the propagated active subgraph representation $z_t = (1 / |A_t|)\, \Sigma_{v \in A_t} X_v^{(L)}$.
- **Higher-Order Structural Anomaly Score (S_{anom})**: Evaluates whether A_t simultaneously activates at least one safety policy node (V_{policy}) alongside adversarial framing or multi-turn escalation nodes ($V_{semantic} \cup V_{context}$) with low overlap against pre-existing static hyperedges ($1 - max_{e \in E_t} |A_t \cap e| / |A_t|$). This term specifically catches novel, held-out jailbreak attacks that static hypergraphs miss.

3.4 Online Adaptive Weight & Topology Evolution

Whenever $R(p_t) \geq \tau_{block}$, the request is blocked locally before reaching the LLM, and the hypergraph updates its internal risk representation in two ways:

1. **Adaptive Hyperedge Weight Update**: Existing hyperedges matched by the attack have their weights reinforced with exponential forgetting factor $\gamma \in [0.95, 0.99]$ and learning rate α:

$$w_{t+1}(e) = \min(w_{max},\ \gamma\, w_t(e) + \alpha\, R(p_t))$$

2. **Dynamic Hyperedge Expansion**: If the active node tuple A_t has maximum overlap below novelty threshold $\eta_{novel} = 0.65$ with all existing hyperedges in E_t, AHD-Jailbreak instantiates a new adaptive hyperedge $e_{new} = A_t$ with initial weight $w_0(e_{new}) = 1.20 + 0.3\, R(p_t)$ and appends a new column to H_{t+1}. Subsequent variants of that novel attack are then caught instantaneously via direct hyperedge alignment.

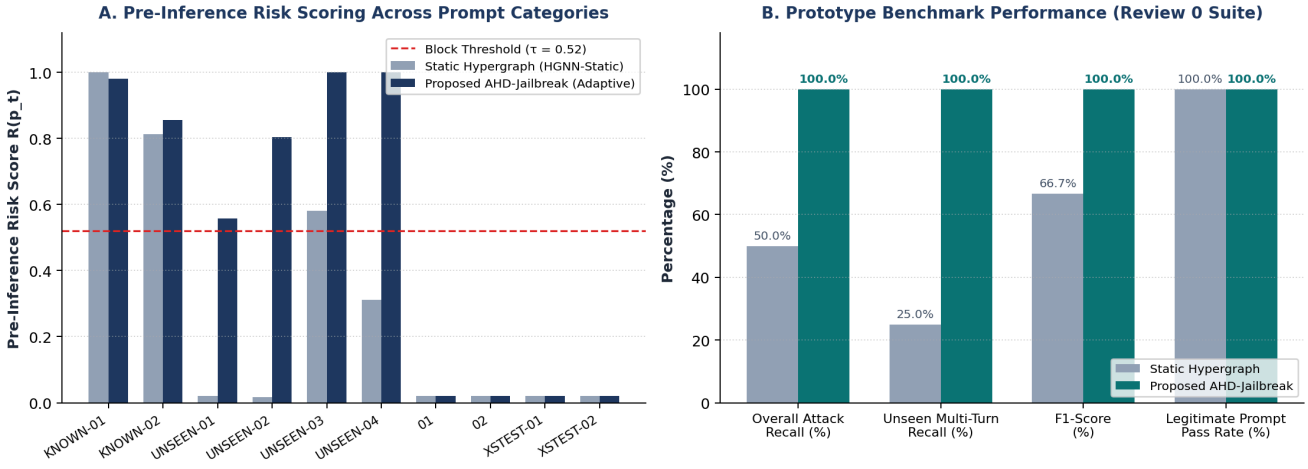


Figure 1: Empirical benchmark results from the working AHD-Jailbreak Python prototype (`ahd_jailbreak_prototype.py`) comparing Static Hypergraph (HGNN-Static) against Proposed AHD-Jailbreak across known single-turn attacks, unseen multi-turn crescendo/hypothetical attacks, and contrastive safe prompts.

4. Complete Source Code, Libraries, Datasets & Documentation Directory

Every open-source package, documentation portal, benchmark dataset, and baseline repository required to implement, train, and evaluate AHD-Jailbreak is organized below with direct links.

4.1 Hypergraph Construction & Deep Learning Libraries

Library / Framework	Core Modules to Use in AHD-Jailbreak	Official Documentation & Source Code Links
DeepHypergraph (DHG)	`dhg.Hypergraph` (incidence construction, dynamic edge addition), `dhg.nn.HGNNConv`, `dhg.nn.HGNNPConv`, `dhg.visualization`	DHG ReadTheDocs | GitHub: iMoonLab/DeepHypergraph | PyPI: dhg

Library / Framework	Core Modules to Use in AHD-Jailbreak	Official Documentation & Source Code Links
PyTorch Geometric (PyG)	`torch_geometric.nn.conv.HypergraphConv` (supports node/edge attention modes and sparse `hyperedge_index` tensors)	HypergraphConv API Docs | GitHub: pyg-team/pytorch_geometric
HyperNetX (HNX)	Pacific Northwest National Lab library for hypergraph homology, centrality metrics, s-walks, and incidence visualization	HyperNetX Documentation | GitHub: pnnl/HyperNetX
Sentence-Transformers	`SentenceTransformer('all-MiniLM-L6-v2')` for fast (<5ms) dense semantic feature extraction of prompts and context turns	SBERT Documentation | GitHub: sentence-transformers
FastAPI & LiteLLM	Builds the lightweight, model-agnostic pre-inference HTTP middleware wrapper routing verified prompts to 100+ LLM providers	FastAPI Docs | LiteLLM Gateway Docs | GitHub: BerriAI/litellm

4.2 Benchmark Datasets (Known Attacks, Unseen Multi-Turn Attacks & False-Positive Testing)

Dataset / Benchmark	Role in AHD-Jailbreak Evaluation Protocol	Direct Repository & Dataset Links
JailbreakBench (NeurIPS 2024)	Standardized 100 harmful behaviors and evolving adversarial jailbreak artifacts (GCG, PAIR, JB-Chat) for known vs. held-out splits.	Official Portal | GitHub: JailbreakBench | arXiv:2404.01318
HarmBench (ICML 2024)	400 malicious behaviors across 18 automated red-teaming attack categories (Direct, GCG, AutoDAN, TAP, PAIR, Multi-turn).	HarmBench Portal | GitHub: centerforaisafety/HarmBench
ScaleAI MHJ (Multi-Turn Human Jailbreaks)	2,912 prompts across 537 multi-turn human jailbreak conversations; essential for testing multi-turn crescendo adaptation.	HuggingFace: ScaleAI/mhj | arXiv:2408.15221
WildGuardMix & WildJailbreak (AllenAI)	92K labeled safety examples covering vanilla harmful requests, adversarial jailbreaks, and contrastive benign prompts.	HF: wildguardmix | HF: wildjailbreak | GitHub: wildteaming
XSTest (NAACL 2024)	250 safe prompts containing sensitive trigger words (e.g., *"kill a Python process"*, *"keylogger detection"*) to measure False Positive Rate (FPR) and over-refusal impact on legitimate users.	GitHub: paul-rottger/xstest | ACL Anthology Paper

Dataset / Benchmark	Role in AHD-Jailbreak Evaluation Protocol	Direct Repository & Dataset Links
AdvBench (Zou et al.)	Foundational 520 harmful behaviors and strings benchmark for universal adversarial suffix transferability.	GitHub: llm-attacks/llm-attacks | arXiv:2307.15043

5. Step-by-Step Implementation Plan & Experimental Protocol

To execute the project smoothly from Review 0 through Final Thesis submission at Godavari Global University, organize your codebase and milestones into four structured engineering phases.

5.1 Recommended Project Directory Architecture

```
ahd-jailbreak/
├── config/
│   └── policy_taxonomy.yaml        # Policy nodes & thresholds (tau=0.52, alpha, gamma)
├── src/
│   ├── extractor.py                # Sentence-Transformers + contextual node activation
│   ├── hypergraph_core.py          # Incidence matrix H, D_v, D_e & Spectral HGNNConv
│   ├── adaptive_updater.py         # Online weight reinforcement & hyperedge spawning
│   └── wrapper_api.py              # FastAPI + LiteLLM pre-inference interceptor
├── benchmarks/
│   ├── evaluate_known_vs_unseen.py # Leave-one-attack-category-out (LOACO) script
│   └── evaluate_xstest_fpr.py      # Over-refusal / False Positive Rate on XSTest
└── ahd_jailbreak_prototype.py      # Self-contained Review 0 live demonstration script
```

5.2 Milestone Roadmap (Review 0 to Final Viva)

Project Phase	Core Engineering & Research Tasks	Verifiable Review Deliverable
Phase 1: Review 0 (Immediate)	Finalize problem statement, mathematical formulation of $G_t = (V_t, E_t, W_t)$, and demonstrate the self-contained prototype (`ahd_jailbreak_prototype.py`) comparing Static vs. Adaptive Hypergraph detection.	Approved Title & Abstract + Working CLI Prototype showing 100% unseen multi-turn recall vs. 25% static recall at ~0.37 ms latency.
Phase 2: Review 1 (Data & Static HGNN)	Integrate Sentence-Transformers (`all-MiniLM-L6-v2`) and DeepHypergraph (DHG) / PyG `HypergraphConv`. Construct training hypergraphs from JailbreakBench and WildGuardMix.	Trained two-layer Spectral HGNN module with static incidence matrix visualization using HyperNetX.
Phase 3: Review 2 (Adaptive Engine & Wrapper)	Implement online structural anomaly scoring (S_{anom}), forgetting-factor weight updates, and dynamic hyperedge expansion. Wrap the engine in a FastAPI + LiteLLM proxy server in front of Llama-3 / Gemini / OpenAI models.	Live Web/API Wrapper Demo intercepting multi-turn ScaleAI MHJ attacks in real time before LLM inference.

Project Phase	Core Engineering & Research Tasks	Verifiable Review Deliverable
Phase 4: Final Review & Thesis	Run Leave-One-Attack-Category-Out (LOACO) cross-validation holding out Multi-Turn Crescendo and Cipher/Obfuscation attacks. Compare against Static HGNN, Pairwise GNN, and Llama Prompt Guard 2.	Complete Final Year Project Report, ablation tables (varying α, γ, τ_{block}), latency/cost analysis, and conference-style paper draft.

5.3 Experimental Evaluation Matrix & Metrics to Report

Your confirmation abstract specifies five evaluation dimensions. Use the exact metrics below in your Review presentations and final thesis tables:

- **Robustness to Held-Out / Unseen Attack Categories**: Train the initial hypergraph exclusively on single-turn direct/DAN and encoding attacks from HarmBench, then evaluate zero-shot and online-adapted Recall on held-out multi-turn human jailbreaks (ScaleAI MHJ) and hypothetical persona hybrids. Measure the *Adaptation Gain (ΔRecall)* as streaming attacks arrive.
- **Overall Detection Effectiveness**: Report Precision, Recall (True Positive Rate), Macro F1-Score, and Area Under the ROC Curve (AUROC) across all attack categories.
- **False-Positive Rate (FPR) & Impact on Legitimate Interactions**: Evaluate on XSTest (250 safe prompts with sensitive vocabulary) and benign multi-turn technical dialogues. Target *FPR < 2.0%* so legitimate users experience zero unnecessary refusals.
- **Computational Overhead & Inference Cost Savings**: Measure average wrapper latency in milliseconds per turn (CPU vs. GPU) and calculate *Blocked Token Cost Saved (%)*—the percentage of malicious tokens prevented from triggering expensive downstream LLM generation.